

Microservices Lab

De un monolito Spring Boot a una arquitectura completa de microservicios con Spring Cloud, Kafka, seguridad OAuth2/JWT y despliegue en Kubernetes

Resumen técnico extendido del curso — conceptos, patrones de diseño, decisiones de arquitectura y diferencias respecto a cómo se hace habitualmente en producción real.

Stack: Spring Boot 4.1.1 · Spring Cloud 2025.1.3 · Angular 22 · PostgreSQL · Apache Kafka (KRaft) · Kubernetes

Índice

Arquitectura general

Mapa completo de componentes

Principios de diseño transversales

Fase 0 — El monolito de partida

Fase 1 — Descomposición en microservicios y base de datos por servicio

Fase 2 — Comunicación entre servicios

Síncrona: Feign

Asíncrona: Kafka, patrón Outbox + Inbox

Fase 3 — Descubrimiento de servicios y configuración centralizada

Fase 4 — API Gateway

Fase 5 — Resiliencia (Resilience4j)

Fase 6 — Observabilidad distribuida

Fase 7 — Seguridad: OAuth2, JWT y PKCE

Fase 8 — Kubernetes sección extendida

Conceptos base de Kubernetes

Docker Compose vs. Kubernetes: qué cambia de verdad

Qué te da Kubernetes "a mayores"

Los manifiestos del proyecto, componente a componente

Los dos bugs de red reales que solo aparecen en Kubernetes

Eureka / Config Server vs. Kubernetes nativo

Catálogo de patrones de diseño usados

Lo que se hizo distinto de cómo se suele hacer en producción real

Cierre: roadmap completo

1. Arquitectura general

1.1 Mapa completo de componentes

El sistema final tiene 10 componentes desplegables: un frontend Angular y 9 procesos backend (6 aplicaciones Spring Boot propias + Kafka + PostgreSQL x2 + Zipkin). Cada microservicio de negocio (`catalog-api` , `order-service`) tiene su propia base de datos — nadie más la toca directamente.

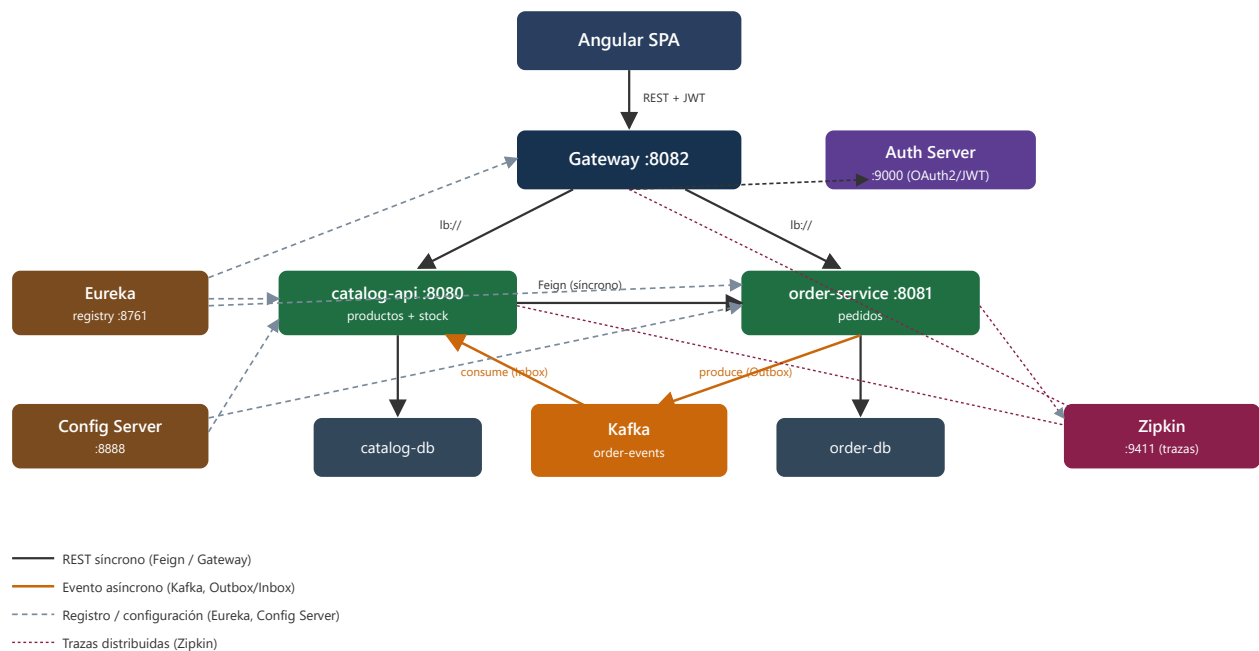


Fig. 1 — Arquitectura completa: 10 componentes, 4 tipos de comunicación.

1.2 Principios de diseño transversales

- **Ningún servicio de negocio conoce la IP de otro.** Todo se resuelve por nombre lógico (Eureka) o por nombre de contenedor/Service (Docker/Kubernetes).
- **Cada servicio es dueño exclusivo de sus datos.** Nadie hace `JOIN` entre bases de datos de servicios distintos; la única forma de "ver" datos de otro servicio es pedirselos por su API o enterarte por un evento.
- **El frontend solo conoce un punto de entrada** (el Gateway) y, para login, al Auth Server directamente — nunca a `catalog-api` u `order-service` a pelo.
- **La configuración vive fuera del código** (Config Server + variables de entorno), para poder cambiar comportamiento sin recompilar.

2. Fase 0 — El monolito de partida

Punto de partida deliberado: un único Spring Boot (`catalog-api`) con datos en memoria y un frontend Angular hablándole directo. Sirve como referencia de "cómo no vamos a estar" y para medir, fase a fase, qué problema resuelve cada pieza que se añade después.

Si en una entrevista te preguntan por qué empezar así: porque descomponer prematuramente sin entender qué dolor resuelve cada pieza (Gateway, discovery, mensajería...) lleva a copiar patrones de microservicios sin necesidad real — el "monolito distribuido".

3. Fase 1 — Descomposición en microservicios y base de datos por servicio

El monolito se separó en `catalog-api` y `order-service`, cada uno con su propio PostgreSQL (`catalog-db` en :5433, `order-db` en :5434), orquestados con `docker-compose`.

PATRÓN: DATABASE PER SERVICE

Cada microservicio tiene su propio esquema/instancia de base de datos, inaccesible directamente para los demás. Es la pieza fundacional de la independencia de despliegue: si nadie más puede leer tu tabla directamente, puedes cambiar su esquema sin coordinarte con otro equipo. El coste es que ya no tienes `JOIN`s ni transacciones ACID entre agregados de servicios distintos — de ahí nace la necesidad del patrón Outbox/Inbox de la Fase 2.

4. Fase 2 — Comunicación entre servicios

4.1 Síncrona: Feign

`order-service` necesita el precio de un producto para crear un pedido, así que le pregunta a `catalog-api` en el momento, vía un cliente declarativo Feign:

```
@FeignClient(name = "catalog-api", path = "/api", fallbackFactory =
CatalogClientFallBackFactory.class)
public interface CatalogClient {
    @GetMapping("/products/{id}")
    ProductResponse getProduct(@PathVariable Long id);
}
```

Nótese que no hay URL: `name = "catalog-api"` es el nombre lógico que Eureka resuelve a una instancia real (ver Fase 3). El `fallbackFactory` se explica en la Fase 5.

4.2 Asíncrona: Kafka — patrón Outbox (productor) + Inbox idempotente (consumidor)

Cuando se crea un pedido, `catalog-api` necesita enterarse para descontar stock — pero esto no debería bloquear la respuesta al usuario ni acoplar la disponibilidad de un servicio a la del otro. Se resuelve con un evento de dominio (`OrderCreated`) publicado en Kafka, tópico `order-events` .

PROBLEMA QUE RESUELVE: EL DUAL-WRITE

Si `OrderService.create()` hiciera `repository.save(order)` y luego `kafkaTemplate.send(...)` directamente, hay una ventana entre ambas operaciones: si el proceso muere justo después del `save` y antes del `send` , el pedido queda guardado pero el evento nunca sale — `catalog-api` nunca descuenta el stock. Escribir en dos sistemas distintos (base de datos + broker) sin una transacción que abarque ambos es inherentemente inseguro.

PATRÓN: TRANSACTIONAL OUTBOX

En vez de publicar directamente en Kafka, `OrderService.create()` guarda el pedido **y** una fila en una tabla `outbox_event` — en la misma transacción JPA:

```
Order created = repository.save(order);
outboxEvent.setPayload(objectMapper.writeValueAsString(
    new OrderCreatedEvent(eventId, created.getId(), created.getProductId(),
    created.getQuantity())));
outboxEventRepository.save(outboxEvent); // misma transacción: o las dos filas, o ninguna
```

Un proceso aparte, `OutboxEventRelay` (`@Scheduled(fixedDelay = 5000)`), va leyendo los eventos no publicados y los manda a Kafka; solo los marca `publishedAt` tras confirmación del broker. Si el `send` falla, simplemente se reintenta en el siguiente ciclo de 5s — nunca se pierde el evento porque su existencia depende solo de la base de datos, no de que Kafka estuviera disponible en el instante exacto del pedido.

PATRÓN: INBOX / IDEMPOTENT CONSUMER

Kafka garantiza *at-least-once*: un rebalanceo de consumidor, un reintento, o un reinicio pueden hacer que `catalog-api` reciba el mismo evento dos veces. Si el consumidor simplemente restara stock cada vez que le llega el mensaje, un duplicado dejaría el stock mal. La solución es una tabla `processed_event` con el `eventId` (UUID) como clave primaria:

```
if (processedEventRepository.existsById(event.getEventId())) {  
    return; // ya procesado, no volver a descontar stock  
}  
product.setStock(product.getStock() - event.getQuantity());  
productRepository.save(product);  
processedEventRepository.save(processedEvent);
```

Este chequeo + el guardado de `product` y de `processed_event` ocurren en la misma transacción (`@Transactional` en `KafkaConsumers`), así que un fallo a mitad de camino no deja el sistema en un estado donde se descontó el stock pero no quedó registrado como procesado (lo que abriría la puerta a reprocesarlo de más).

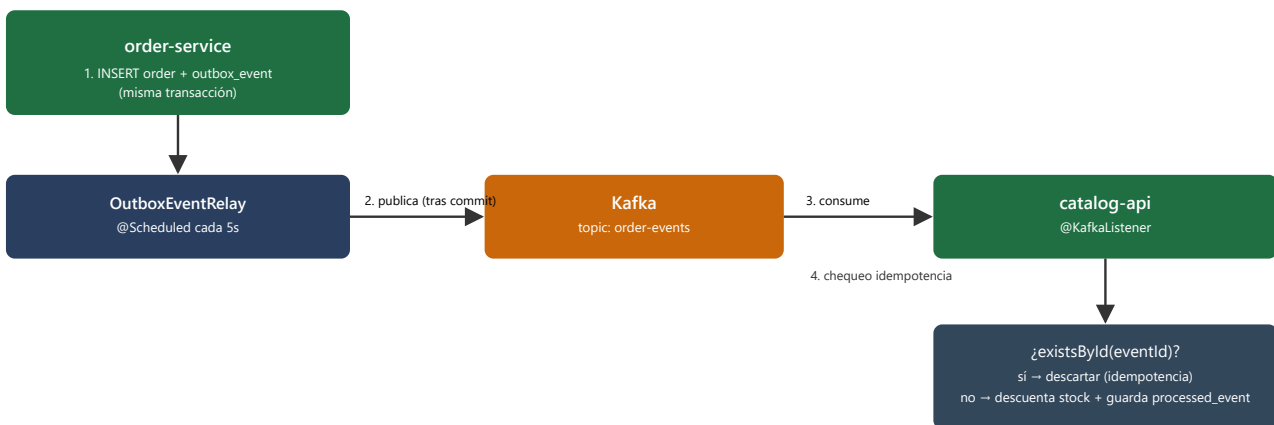


Fig. 2 — Outbox (garantiza que el evento siempre sale) + Inbox (garantiza que procesarlo dos veces no rompe nada).

DETALLE PARTICULAR

No se usa *exactly-once* de Kafka (transacciones de productor/consumidor de Kafka) — se optó deliberadamente por el patrón Outbox/Inbox a nivel de aplicación, que es más portable (funciona igual con cualquier broker) y más fácil de razonar/depurar para aprenderlo, aunque añade la tabla y el `relay` como piezas extra.

5. Fase 3 — Descubrimiento de servicios y configuración centralizada

PATRÓN: SERVICE DISCOVERY (CLIENT-SIDE) — EUREKA

Antes de esta fase, Feign apuntaba a una URL fija de `catalog-api`. A partir de aquí, cada instancia se registra en Eureka al arrancar (`eureka.client.service-url.defaultZone`) y Feign/Gateway resuelven por nombre lógico con el prefijo `lb://` (*load-balanced*). Es **client-side discovery**: la lógica de "a qué instancia concreta le hablo" vive en el cliente (Feign/Gateway con su librería de balanceo), no en un balanceador de infraestructura aparte — a diferencia del *server-side discovery* que ofrece de forma nativa un `Service` de Kubernetes (ver Fase 8).

PATRÓN: EXTERNALIZED CONFIGURATION — SPRING CLOUD CONFIG

La configuración común a todos los servicios (URL de Eureka, `bootstrap-servers` de Kafka, endpoint de Zipkin) vive en `config-repo/application.yml`, servida por `config-server`. Cada servicio la pide al arrancar con:

```
spring:
  config:
    import: optional:configserver:http://localhost:8888
```

El prefijo `optional:` es importante: sin él, si el Config Server no responde, el arranque del servicio **se detiene por completo**; con él, el servicio sigue arrancando con lo que ya tenga localmente. Además existe `application-docker.yml`, que sobrescribe solo las claves que cambian dentro de Docker/Kubernetes (nombres de contenedor/Service en vez de `localhost`), activado por el perfil `docker`.

DETALLE PARTICULAR

El Config Server usa **backend nativo** (lee los YAML de una carpeta local montada, no de un repositorio Git remoto). En producción real, lo habitual es un backend Git (a menudo el mismo repo del código, o uno dedicado a configuración) para tener versionado, historial y poder hacer *rollback* de un cambio de configuración igual que de código. Aquí se simplificó para el aprendizaje, pero es la primera pregunta que un entrevistador senior haría sobre esta pieza.

6. Fase 4 — API Gateway

El Gateway es el único punto de entrada del frontend. Enruta por prefijo de path hacia el nombre lógico del servicio (resuelto por Eureka):

```
spring.cloud.gateway.server.webmvc.routes:
- id: catalog-api
  uri: lb://catalog-api
  predicates: [{ name: Path, args: { pattern: /api/products/** } }]
- id: order-service
  uri: lb://order-service
  predicates: [{ name: Path, args: { pattern: /api/orders/** } }]
```

PATRÓN: API GATEWAY

Centraliza en un solo sitio lo que, sin él, tendría que repetirse en cada microservicio: enrutamiento, CORS, y (desde la Fase 7) validación de JWT. El frontend deja de necesitar conocer cuántos microservicios hay o dónde vive cada uno — solo conoce el Gateway.

DETALLE PARTICULAR — LO DE WEBFLUX

Spring Cloud Gateway, por defecto y en el 99% de los tutoriales/producción, es reactivo: corre sobre Spring WebFlux y un servidor Netty no bloqueante. Este proyecto usa deliberadamente la variante **Server MVC** (`spring-cloud-starter-gateway-server-webmvc`), que es servlet/blocking sobre Tomcat — la misma familia de programación que `catalog-api` y `order-service` (ambos `spring-boot-starter-webmvc`).

Por qué importa la diferencia: un Gateway reactivo puede atender muchísimas más conexiones concurrentes con pocos hilos, porque nunca bloquea un hilo esperando una respuesta de red — usa E/S no bloqueante y un modelo de *Mono/Flux*. Es la opción natural cuando el Gateway va a ser un cuello de botella de alto tráfico. La contrapartida es una curva de aprendizaje más dura (programación reactiva, depuración de *stack traces* asíncronos, cuidado especial si alguna librería que uses es bloqueante) y que mezclar código bloqueante ahí dentro por error puede colgar el *event loop*.

Por qué se eligió aquí la variante MVC: mantiene todo el proyecto en un único modelo de programación (bloqueante, imperativo, el que ya usas en Spring MVC clásico), lo cual es mejor para *aprender* el patrón de Gateway sin tener que aprender programación reactiva al mismo tiempo. Para un proyecto de portafolio/entrevista, es perfectamente defendible — pero hay que saber decir en la entrevista "esto normalmente se hace con WebFlux, aquí usé Server MVC a propósito por X razón", no dar por hecho que es lo estándar.

7. Fase 5 — Resiliencia con Resilience4j

La llamada síncrona de `order-service` a `catalog-api` (Fase 2) es el punto donde un microservicio caído podría arrastrar a otro. Se le añaden tres mecanismos combinados, todos configurados en `application.yaml` de `order-service`:

```
resilience4j:
  timelimiter:
    instances.catalog-api: { timeout-duration: 2s }
  retry:
    instances.catalog-api: { max-attempts: 3, wait-duration: 200ms }
  circuitbreaker:
    instances.catalog-api:
      sliding-window-size: 10
      failure-rate-threshold: 50
      wait-duration-in-open-state: 10s
      permitted-number-of-calls-in-half-open-state: 3
```

PATRÓN: TIMEOUT + RETRY + CIRCUIT BREAKER

- **TimeLimiter:** si `catalog-api` no responde en 2s, se corta la espera — nunca un hilo bloqueado indefinidamente.
- **Retry:** hasta 3 intentos con 200ms de espera entre ellos, para absorber fallos transitorios (un GC pause, un blip de red).
- **Circuit Breaker:** si de las últimas 10 llamadas (ventana deslizante) el 50% falla, el circuito se *abre* durante 10s — deja de intentar llamar a `catalog-api` en absoluto (falla rápido, sin ni siquiera intentar la red) y prueba de nuevo con 3 llamadas de sondeo (*half-open*) pasado ese tiempo. Evita que un servicio caído reciba además una avalancha de reintentos que le impidan recuperarse.

PATRÓN: FALLBACK DIFERENCIADO POR CAUSA (FALLBACKFACTORY)

Un fallback ingenuo trataría "producto no encontrado" (404, error de negocio legítimo) igual que "catalog-api no responde" (fallo de infraestructura) — sería un error de diseño grave devolver "servicio no disponible" cuando en realidad el producto simplemente no existe. El `FallbackFactory` recibe la `Throwable` causante y distingue:

```
public CatalogClient create(Throwable cause) {
    return id -> {
        if (cause instanceof FeignException.NotFound) {
            throw new NoSuchElementException("Producto no encontrado: " + id); // 404 real
        }
        throw new CatalogUnavailableException("Catalog unavailable, try again later."); //
        circuito abierto / timeout
    };
}
```

DETALLE DE NOMENCLATURA (VERSIÓN "FUTURA" DE SPRING CLOUD)

La propiedad para activar el circuit breaker en OpenFeign es aquí `spring.cloud.openfeign.circuitbreaker.enabled: true`. En versiones más antiguas de Spring Cloud (y en mucha documentación/tutoriales que vas a encontrar buscando) la propiedad equivalente era `feign.circuitbreaker.enabled`. Si ves ambas en distintas fuentes, no es un error — es que el proyecto usa Spring Cloud 2025.1.3, una versión muy reciente/"futura" respecto a la mayoría de tutoriales en internet, con varios artefactos y propiedades renombrados. Vale la pena mencionar esto en entrevista: demuestra que sabes leer *release notes* y no memorizas nombres de propiedades a ciegas.

8. Fase 6 — Observabilidad distribuida

Micrometer Tracing + Zipkin instrumentan automáticamente peticiones HTTP y llamadas Feign: cada petición entrante genera un `traceId` que se propaga por cabeceras a través del Gateway, `order-service` y `catalog-api`, visible como una traza completa en Zipkin (<http://localhost:9411>).

```
management:
  tracing.sampling.probability: 1.0 # 100% en dev; en prod real se baja (ej. 0.1) para no saturar Zipkin
  zipkin.tracing.endpoint: http://localhost:9411/api/v2/spans
```

PATRÓN: CORRELATION ID MANUAL — DONDE LA TRAZA AUTOMÁTICA SE ROMPE

La propagación automática de `traceId` / `spanId` funciona mientras la llamada es síncrona (HTTP/Feign, en el mismo hilo lógico). En cuanto el flujo cruza una frontera *asíncrona* — el `@Scheduled` del `OutboxEventRelay`, o el `@KafkaListener` del consumidor — ese contexto de traza se pierde: son hilos/invocaciones completamente distintos, sin relación con la petición HTTP original que generó el pedido.

La solución aquí es **correlación manual por `eventId`** (el UUID de negocio del evento, no un `traceId` técnico), metido en el MDC de SLF4J en cada punto relevante:

```
MDC.put("eventId", eventId.toString());
log.info("Pedido {} creado...", created.getId());
// ...
MDC.clear();
```

Con el patrón de log `"[%X{traceId:-},%X{spanId:-},%X{eventId:-}] "`, puedes *grep*-ear un `eventId` concreto a través de los logs de `order-service` (creación + publicación del outbox) y de `catalog-api` (consumo) aunque Zipkin nunca haya visto esa cadena como una sola traza.

9. Fase 7 — Seguridad: OAuth2, JWT y PKCE

9.1 Auth Server con autoconfiguración por propiedades

El servidor de autorización usa `spring-boot-starter-security-oauth2-authorization-server` configurado enteramente por propiedades (sin una clase Java de configuración manual con `OAuth2AuthorizationServerConfigurer`, que era la forma habitual en versiones anteriores):

```
spring.security.oauth2.authorizationserver:
  issuer: http://localhost:9000
  client.angular-spa:
    registration:
      client-id: angular-spa
      client-authentication-methods: [none]      # cliente público, sin secreto
      authorization-grant-types: [authorization_code, refresh_token]
      redirect-uris: [http://localhost:4200/callback]
      scopes: [openid, profile]
    require-proof-key: true                      # obliga PKCE
    require-authorization-consent: false
```

PATRÓN: AUTHORIZATION CODE + PKCE (PARA CLIENTES PÚBLICOS)

Un *client secret* tradicional no se puede usar en una SPA: es JavaScript que corre en el navegador del usuario, cualquiera puede leerlo con las herramientas de desarrollador. PKCE (*Proof Key for Code Exchange*) sustituye ese secreto fijo por uno de un solo uso generado en el propio navegador:

1. Angular genera un `code_verifier` aleatorio y su hash SHA-256, el `code_challenge`.
2. Pide login enviando el `code_challenge` (no el verifier).
3. Al canjear el código de autorización por el token, envía el `code_verifier` original.
4. El Auth Server comprueba que el hash del verifier coincide con el challenge recibido al principio.

Si alguien interceptase el *authorization code* a mitad de camino (por ejemplo en el historial del navegador o un proxy), no podría canjearlo por un token sin conocer también el `code_verifier`, que nunca viaja hasta ese último paso.

9.2 Flujo completo

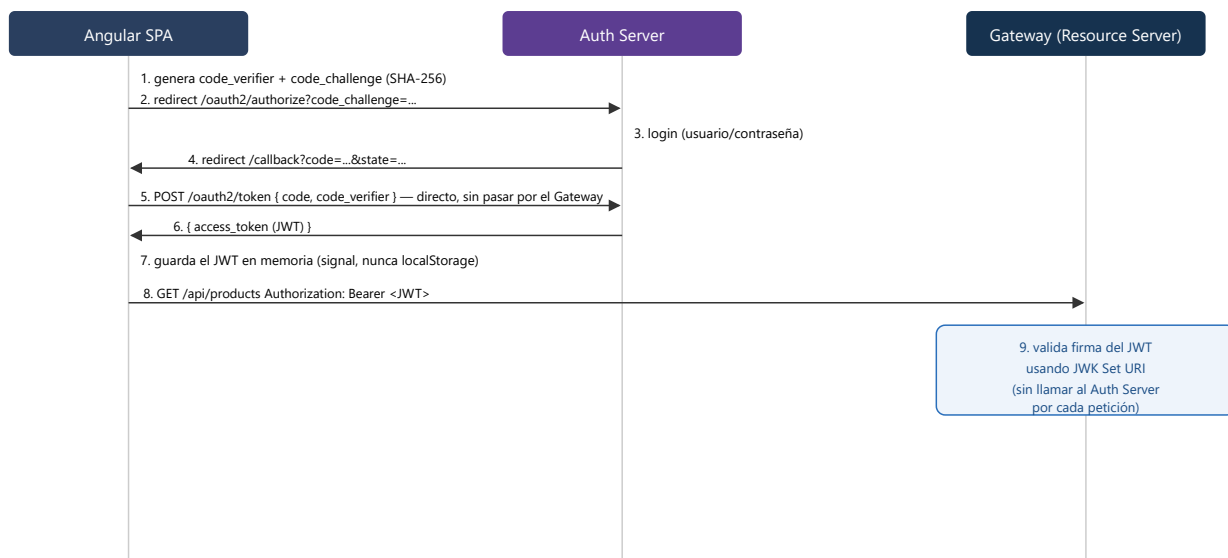


Fig. 3 — Authorization Code + PKCE, y validación stateless del JWT en el Gateway.

PATRÓN: RESOURCE SERVER STATELESS (VALIDACIÓN JWT POR JWK SET URI)

El Gateway no llama al Auth Server en cada petición para saber si un token es válido. Descarga (y cachea) el conjunto de claves públicas del Auth Server (`/oauth2/jwks`) y verifica la firma del JWT localmente — el token es autocontenido (*self-contained*): su firma y sus *claims* (expiración, *issuer*, *scopes*) bastan para validarlo sin ida y vuelta a ningún sitio. Esto es lo que hace a JWT escalar mejor que las sesiones tradicionales en un sistema distribuido — al coste de no poder revocar un token individual antes de que expire por sí solo (ver sección 11).

DETALLE PARTICULAR: CORS Y EL ORDEN DE LOS FILTROS

Angular llama a `/oauth2/token` directamente desde el navegador. Un `WebMvcConfigurer` normal para CORS (el que sirve en `catalog-api / order-service`) no es suficiente aquí: **Spring Security intercepta la petición antes de que llegue a la capa de Spring MVC** donde ese CORS se aplicaría, y para el *preflight* OPTIONS anónimo responde con un redirect a `/login` — el navegador nunca ve las cabeceras CORS y aborta antes de intentar el POST real. La solución fue un `CorsFilter` registrado con `Ordered.HIGHEST_PRECEDENCE` , que se ejecuta *antes* que el filtro de Spring Security y resuelve el *preflight* él mismo, sin tocar la cadena de seguridad autoconfigurada. Es un ejemplo real de por qué el orden de la cadena de filtros de Spring Security importa más de lo que parece a primera vista.

ALTERNATIVA MÁS SEGURA, NO IMPLEMENTADA: BFF (BACKEND FOR FRONTEND)

Guardar el token en memoria del SPA (aunque nunca en `localStorage`) sigue exponiéndolo a un XSS mientras la página está viva. El patrón **BFF** evita que el navegador vea el token JWT en absoluto: un backend intermedio (a menudo el propio Gateway) hace el intercambio OAuth2 por detrás y le da al navegador solo una cookie de sesión `HttpOnly` (inaccesible desde JavaScript). Es más seguro pero añade una pieza más de infraestructura y estado — se optó por el flujo PKCE puro en el SPA por ser más simple de enseñar/aprender primero, mencionando el BFF como la evolución natural hacia producción real.

10. Fase 8 — Kubernetes

Esta es la fase donde vale la pena pararse más: es infraestructura de verdad y donde suele haber más lagunas en una entrevista.

10.1 Conceptos base de Kubernetes

Objeto	Qué es	Equivalente/analogía en Docker Compose
Cluster	Conjunto de máquinas (nodos) gestionadas como una sola unidad de cómputo.	No existe — Compose es siempre un único host.
Node	Una máquina (física o VM) del clúster, con su Kubelet y su runtime de contenedores.	El host único donde corre Docker Engine.
Pod	La unidad mínima desplegable: uno o más contenedores que comparten red y almacenamiento. En este proyecto, 1 contenedor por Pod.	Un contenedor de <code>docker-compose.yml</code> .
Deployment	Declara "quiero N réplicas de este Pod, siempre" y gestiona el <i>rollout</i> de nuevas versiones sin downtime.	<code>docker compose up --scale</code> , pero declarativo y autorreparable.
Service	Una IP virtual estable + nombre DNS que balancea tráfico entre todos los Pods sanos que coincidan con su <i>selector</i> de etiquetas — sin importar en qué nodo estén ni cuántas réplicas haya en cada momento.	El nombre de contenedor/ <code>container_name</code> en la red de Compose — pero ahí resuelve a una IP real de un único contenedor, aquí a una IP virtual que reparte entre N Pods.
ConfigMap	Configuración no sensible (ficheros o variables) inyectable en Pods.	Un <code>volumes:</code> bind-mount de un fichero de config, o variables en <code>environment:</code> .
Secret	Igual que ConfigMap pero pensado para credenciales — <i>qjg</i> : solo están codificadas en base64 por defecto, no cifradas , salvo que actives cifrado en reposo (<i>encryption at rest</i>) o uses una solución externa (Vault, Sealed Secrets).	<code>environment:</code> con valores en claro en el propio <code>docker-compose.yml</code> .
PersistentVolumeClaim (PVC)	Una petición de almacenamiento durable que sobrevive a que el Pod se destruya y se recree.	Un <code>volumes:</code> nombrado de Compose (<code>catalog-db-data</code>).
Ingress	Puerta de entrada HTTP declarativa desde fuera del clúster hacia Services internos, con enrutamiento por host/path.	El <code>ports:</code> <code>"8082:8082"</code> publicado al host.

10.2 Docker Compose vs. Kubernetes: qué cambia de verdad

El malentendido más común: pensar que "meter la app en contenedores y separarla en microservicios" ya te da alta disponibilidad. **No es así.** Con `docker-compose`, por muchos contenedores que tengas, todos corren sobre el mismo Docker Engine, en la misma máquina física. Si esa máquina se apaga, se queda sin disco, o el propio Docker Engine se cae, **pierdes todo** — exactamente igual que si fuera un monolito con un solo proceso. Compose reparte procesos, no reparte el riesgo de la máquina que los aloja.

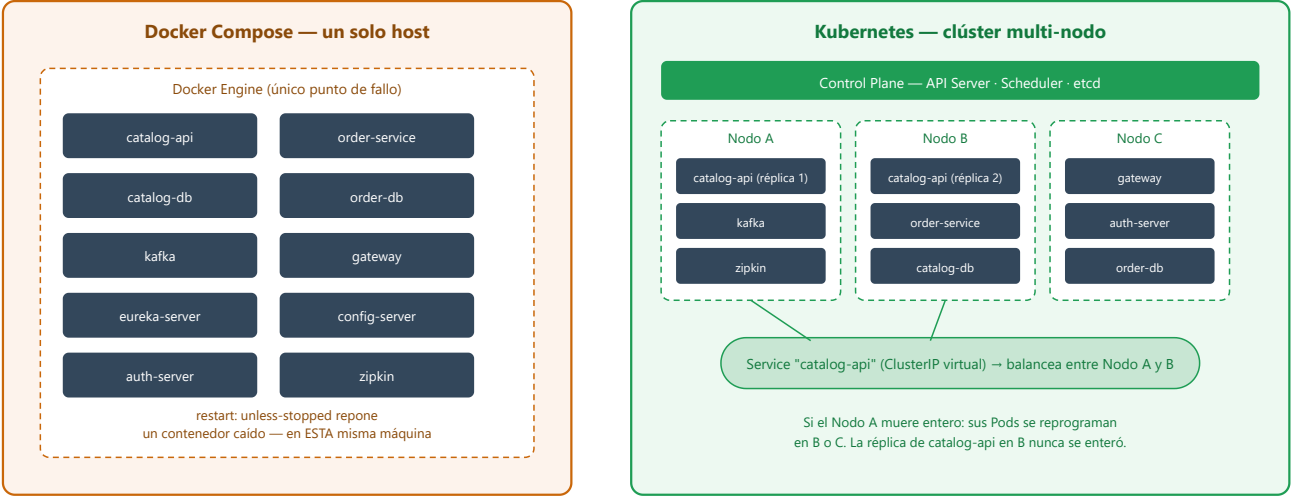


Fig. 4 — Un host frente a un clúster: la diferencia es tolerancia a fallo de máquina completa, no solo de proceso.

Capacidad	Docker Compose	Kubernetes
Reiniciar un contenedor caído	Sí, con <code>restart: unless-stopped</code> — en la misma máquina	Sí (<i>self-healing</i>), y puede recrearse en cualquier nodo con capacidad
Múltiples réplicas de un servicio	Sí, <code>--scale</code> , pero limitado por los recursos de un único host y por conflictos de <code>container_name</code> /puertos fijos	Sí, declarativo (<code>replicas: N</code>), repartidas entre nodos distintos
Tolerancia a fallo de máquina completa	No — es la limitación estructural, no hay a dónde reprogramar nada	Sí — es la razón de ser del clúster multi-nodo
Balanceo de carga entre réplicas	Solo si algo externo lo hace (aquí, indirectamente, Eureka+Gateway a nivel de aplicación)	Nativo, a nivel de plataforma, vía <code>Service</code> — no depende de que la app tenga su propio cliente de discovery
Actualizar una versión sin downtime	Manual / no garantizado	<i>Rolling update</i> nativo (<code>kubectl rollout restart</code>) — sustituye Pods de uno en uno mientras el <code>Service</code> sigue sirviendo con los que quedan
Reconciliación de estado	Imperativo: tú corres el comando cuando algo cambia	Declarativo con <i>control loop</i> : declaras el estado deseado y un controlador lo mantiene así de forma continua
Autoescalado según métricas (CPU/memoria/custom)	No existe de forma nativa	<i>HorizontalPodAutoscaler</i> (no usado en este proyecto, pero es la pieza natural siguiente)

10.3 Qué te da Kubernetes "a mayores" — resumen para entrevista

LO ESENCIAL

- **Un pool compartido de máquinas, no una máquina fija por servicio.** El *scheduler* decide dinámicamente en qué nodo va cada Pod según qué máquina tiene CPU/memoria libres en ese momento (*bin packing*) — no es "cada servicio a su servidor dedicado" ni "todo el stack clonado en cada máquina".
- **Reprogramación ante fallo de nodo completo.** Si un nodo entero muere, sus Pods se recrean en otro nodo vivo automáticamente — la garantía que Compose no puede dar por definición.
- **Anti-affinity entre réplicas.** Las N réplicas de un mismo Deployment tienden a repartirse en nodos distintos, así que perder un nodo no te deja sin ningún ejemplar sano de ese servicio.
- **Service discovery + balanceo a nivel de plataforma,** no de librería cliente: cualquier proceso (sea Java, Python, o un script de shell con `curl`) se beneficia del DNS interno y del balanceo del `Service`, sin necesitar un cliente de Eureka.
- **Declarativo con reconciliación continua.** No ejecutas pasos, describes el resultado que quieres (`replicas: 3`) y un controlador lo hace realidad y lo mantiene así para siempre, no solo la primera vez.
- **Extensible:** *Custom Resource Definitions, Operators, HorizontalPodAutoscaler, service mesh* (Istio/Linkerd) — un ecosistema completo que se apoya en el mismo modelo de objetos declarativos.
- **Portabilidad:** el mismo YAML corre (con ajustes menores) en Minikube local, EKS, GKE, AKS o un clúster on-prem.

10.4 Los manifiestos del proyecto, componente a componente

Fichero	Contenido
00-namespace.yaml	Namespace <code>microservices-lab</code> — aísla lógicamente todos los objetos del proyecto dentro del clúster.
01-secrets.yaml	Credenciales de ambas bases de datos, vía <code>stringData</code> (Kubernetes las guarda codificadas en base64 automáticamente).
02/03-*-db.yaml	Por cada base de datos: <code>PersistentVolumeClaim</code> (1Gi) + <code>Deployment</code> (postgres:16, credenciales vía <code>envFrom.secretRef</code>) + <code>Service</code> <code>ClusterIP</code> .
04-kafka.yaml	Un solo broker KRaft — ver 10.5, es el fichero con más "cicatrices" de depuración real.
05-zipkin.yaml	<code>Deployment</code> + <code>Service</code> simples, imagen pública.
06-eureka-server.yaml / 08-config-server.yaml / 09-auth-server.yaml	Igual que en <code>Compose</code> — casi sin cambios, gracias al DNS de Kubernetes (ver 10.6). El Config Server monta el <code>config-repo</code> como <code>ConfigMap</code> en vez de <code>volumes: bind-mount</code> .
10-catalog-api.yaml / 11-order-service.yaml	Credenciales de BD vía <code>valueFrom.secretKeyRef</code> (nunca en claro en el manifiesto); <code>SPRING_CONFIG_IMPORT</code> apuntando al <code>Service</code> <code>config-server</code> .
12-gateway.yaml	Además del <code>Deployment/Service</code> , incluye un <code>Ingress</code> (<code>ingressClassName: nginx</code>) como demostración de la puerta de entrada HTTP declarativa — la verificación real del proyecto se hizo con <code>kubect1 port-forward</code> por simplicidad.

10.5 Los dos bugs de red reales que solo aparecen en Kubernetes

BUG 1 — NO PUEDES HACER BIND A LA IP VIRTUAL DE TU PROPIO SERVICE

En `docker-compose`, el nombre de un contenedor (`kafka`) resuelve a la **IP real** de ese contenedor — se puede abrir un socket de escucha ahí sin problema. En Kubernetes, el nombre de un `Service` resuelve a una **IP virtual** gestionada por `kube-proxy` vía `iptables` / `IPVS`, que no existe como interfaz de red real dentro del Pod — Kafka fallaba con `BindException: Address not available` al copiar la config de `Compose` tal cual. La corrección: escuchar en `0.0.0.0` (todas las interfaces reales del Pod) y dejar que `KAFKA_ADVERTISED_LISTENERS` (lo que ven los *clientes* al conectarse) siga usando el nombre del `Service`, que para eso sí sirve.

```
KAFKA_LISTENERS:
"PLAINTEXT://0.0.0.0:19092,CONTROLLER://0.0.0.0:19093,PLAINTEXT_HOST://0.0.0.0:9092"
KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:19092,PLAINTEXT_HOST://localhost:9092"
```

BUG 2 — HAIRPIN NAT EN EL CONTROLLER DE KRAFT

Con `KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:19093`, el propio proceso de Kafka intentaba conectarse a la IP virtual de **su propio** Service para el registro del *controller* Raft — un Pod conectando de vuelta a su propia IP virtual de Service es un caso límite ("hairpin NAT") que algunas redes de Kubernetes no manejan bien, y aquí colgaba con timeouts de conexión. Para un nodo único, el controller es el mismo proceso, así que apuntar a `localhost` en vez del nombre del Service evita el problema del todo:

```
KAFKA_CONTROLLER_QUORUM_VOTERS: "1@localhost:19093"
```

Se verificó forzando un reinicio limpio del Pod (`kubectl delete pod -l app=kafka`) y confirmando logs estables durante un rato prolongado.

10.6 Por qué casi no hubo que tocar la configuración de las apps

Como `docker-compose.yml` ya usaba nombres de contenedor en vez de IPs para que los servicios se encontraran, y Kubernetes también resuelve por nombre de `Service` vía su propio DNS interno, las mismas variables de entorno (`SPRING_DATASOURCE_URL=jdbc:postgresql://catalog-db:5432/...`, etc.) funcionan sin cambios. Migrar de verdad fue sobre todo traducir cada servicio de `docker-compose.yml` a un par `Deployment` + `Service` homónimo. Es la mejor prueba práctica de por qué usar nombres lógicos en vez de IPs desde el primer día es una buena práctica que paga dividendos después.

10.7 — EUREKA / CONFIG SERVER VS. KUBERNETES NATIVO

Punto de discusión de entrevista relevante: en un despliegue puramente Kubernetes, el `Service` con su DNS ya te da *service discovery*, y `ConfigMap` / `Secret` ya te dan configuración externalizada — las mismas dos cosas que Eureka y Config Server resuelven a nivel de aplicación. En una arquitectura pensada para correr **solo** en Kubernetes, sería razonable prescindir de ambos y apoyarse enteramente en las primitivas nativas del clúster, simplificando el sistema. En este proyecto se mantuvieron a propósito (no se refactorizó) porque: (a) el objetivo era aprenderlos como piezas de Spring Cloud en sí mismas, y (b) mantenerlos permite que las mismas imágenes/código corran igual en Docker Compose y en Kubernetes sin bifurcar la configuración de la aplicación — un trade-off consciente de portabilidad frente a simplicidad, no un descuido.

11. Catálogo de patrones de diseño usados

Patrón	Problema que resuelve	Dónde se usó
Database per Service	Acoplar el esquema de datos de un servicio al de otro impide desplegarlos de forma independiente	catalog-db / order-db separadas
API Gateway	El frontend no debería conocer ni la topología ni la cantidad de microservicios	Gateway (Fase 4)
Service Discovery (client-side)	URLs fijas se rompen en cuanto una instancia cambia de IP o se escala	Eureka + <code>lb://</code> (Fase 3)
Externalized Configuration	Config hardcodeada obliga a recompilar para cambiar un valor por entorno	Config Server (Fase 3)
Circuit Breaker + Retry + Timeout	Un servicio caído no debe arrastrar a quien lo llama, ni recibir una avalancha de reintentos mientras se recupera	Resilience4j en Feign (Fase 5)
Fallback diferenciado por causa	No confundir un error de negocio (404) con una indisponibilidad de infraestructura	<code>FallbackFactory</code> (Fase 5)
Transactional Outbox	Escribir en BD y publicar en un broker no son atómicos por separado (dual-write)	<code>OutboxEvent</code> + <code>relay @Scheduled</code> (Fase 2)
Inbox / Idempotent Consumer	Kafka entrega <i>at-least-once</i> : un evento duplicado no debe aplicarse dos veces	<code>ProcessedEvent</code> por <code>eventId</code> (Fase 2)
Correlation ID manual	La traza automática se rompe al cruzar fronteras asíncronas (colas, <i>schedulers</i>)	<code>eventId</code> en MDC (Fase 6)
Authorization Code + PKCE	Un cliente público (SPA) no puede guardar un <i>client secret</i> de forma segura	Angular ↔ Auth Server (Fase 7)
Resource Server stateless (JWT)	Validar cada petición sin ida y vuelta al servidor de autorización	Gateway + JWK Set URI (Fase 7)
BFF (discutido, no implementado)	Evitar que el navegador vea el token en absoluto	Alternativa a PKCE puro (Fase 7)

12. Lo que se hizo distinto de cómo se suele hacer en producción real

Sección pensada explícitamente para defender en entrevista: saber señalar tú mismo dónde se simplificó algo, y por qué, vale más que no mencionarlo y que te lo señalen a ti.

Decisión en este proyecto	Lo habitual en producción real	Por qué se simplificó aquí
Spring Cloud Gateway variante Server MVC (servlet, bloqueante)	Spring Cloud Gateway reactivo sobre WebFlux/Netty	Mantener todo el stack en un único modelo de programación (bloqueante) mientras se aprenden los conceptos de Gateway, sin sumar programación reactiva a la vez
Config Server con backend nativo (carpeta local)	Backend Git remoto, versionado	Simplicidad de configuración local para el laboratorio; en producción real se perdería historial/rollback de config
Kafka de un solo broker (KRaft, un nodo)	Clúster de 3+ brokers con factor de replicación > 1	Con un solo broker no hay tolerancia a fallo del propio Kafka — <code>OFFSETS_TOPIC_REPLICATION_FACTOR=1</code> sería impensable en producción
PostgreSQL de una sola instancia , sin réplica	Alta disponibilidad (ej. Patroni, o un gestionado como RDS Multi-AZ)	Simplicidad; en Kubernetes se usó <code>Deployment</code> +PVC en vez de un <code>StatefulSet</code> , válido solo por ser una única réplica
Outbox/Inbox a nivel de aplicación	A veces se usan transacciones nativas del broker (<i>exactly-once</i> de Kafka) o un <i>Change Data Capture</i> (Debezium) sobre la tabla outbox	Más portable y más fácil de entender/depurar para aprender el patrón desde cero
JWT sin mecanismo de revocación	Listas de revocación, introspección, o tokens de vida muy corta + <i>refresh</i> rotativo con detección de reuso	Trade-off inherente a JWT stateless; no se implementó revocación activa
Token de acceso en memoria del SPA	Patrón BFF con cookie <code>HttpOnly</code> , para que el navegador nunca vea el JWT	Menor complejidad de infraestructura para enseñar el flujo PKCE de punta a punta primero
Kubernetes Secrets en texto plano (<code>stringData</code>)	Cifrado en reposo, o gestores externos (HashiCorp Vault, Sealed Secrets, External Secrets Operator)	Foco puesto en el modelo de objetos de Kubernetes, no en gestión de secretos
<code>Deployment</code> +PVC para las bases de datos	<code>StatefulSet</code> (identidad de red estable, volúmenes por réplica) cuando hay más de una réplica	Con una sola réplica de Postgres, un <code>Deployment</code> normal es suficiente y más simple
Verificación con <code>kubectl port-forward</code>	<code>Ingress</code> + <code>LoadBalancer</code> / <code>minikube tunnel</code> como única puerta de entrada real	El <code>Ingress</code> se dejó declarado y documentado, pero <code>port-forward</code> fue más simple para las pruebas del laboratorio
Spring Boot 4.1.1 / Spring Cloud 2025.1.3	La mayoría de proyectos reales en el mercado hoy corren versiones bastante anteriores	No fue una elección deliberada de "hacerlo distinto" sino la versión disponible al momento — pero expone artefactos y propiedades ya renombrados (ver Fase 4 y 5) que vale la pena saber leer en <i>release notes</i> en vez de memorizar nombres fijos

13. Cierre: roadmap completo

Fase	Qué se añadió
0	Monolito de referencia
1	Descomposición en microservicios + base de datos por servicio
2	Comunicación síncrona (Feign) y asíncrona (Kafka, Outbox/Inbox)
3	Eureka (descubrimiento) + Spring Cloud Config (configuración centralizada)
4	API Gateway (Spring Cloud Gateway, variante Server MVC)
5	Resiliencia: timeout + retry + circuit breaker (Resilience4j) con fallback diferenciado
6	Observabilidad: trazas distribuidas (Zipkin) + correlación manual por evento
7	Seguridad: OAuth2/JWT con Spring Authorization Server, PKCE en el cliente Angular
8	Despliegue: Docker Compose completo + manifiestos de Kubernetes para Minikube

Documento generado como resumen técnico del proyecto *microservices-lab* — para preparación de entrevistas técnicas sobre arquitectura de microservicios.